OpenMP (Open Multi-Processing) Library

1. Introduction

OpenMP (Open Multi-Processing) is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran. It allows developers to easily write parallel applications that utilize all available CPU cores. OpenMP provides compiler directives, library routines, and environment variables to control parallel execution.

2. Key Features of OpenMP

- Simple syntax using compiler directives (#pragma).
- Supports shared-memory systems.
- Allows easy control over threads and data sharing.
- Incremental parallelization of existing code.
- Portable and scalable across many architectures.
- Enables synchronization, reductions, and workload distribution.

3. Example Code Using OpenMP

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define SIZE 10000000

int main() {
    int i;
    double start_time, end_time;
    double *A, *B, *C_seq, *C_parallel;

    A = (double *)malloc(SIZE * sizeof(double));
    B = (double *)malloc(SIZE * sizeof(double));
    C_seq = (double *)malloc(SIZE * sizeof(double));
```

```
    C_parallel = (double *)malloc(SIZE *
sizeof(double));

    for (i = 0; i < SIZE; i++) {
        A[i] = i * 0.5;
        B[i] = i * 2.0;
    }

    start_time = omp_get_wtime();
    for (i = 0; i < SIZE; i++) {
        C_seq[i] = A[i] + B[i];
    }
    end_time = omp_get_wtime();
    printf("Sequential addition time: %.5f seconds\n",
end_time - start_time);

    start_time = omp_get_wtime();
    #pragma omp parallel for
    for (i = 0; i < SIZE; i++) {
        C_parallel[i] = A[i] + B[i];
    }
    end_time = omp_get_wtime();
    printf("Parallel addition time (OpenMP): %.5f
seconds\n", end_time - start_time);

    int errors = 0;
    for (i = 0; i < SIZE; i++) {
        if (C_seq[i] != C_parallel[i]) errors++;
    }

    if (errors == 0)
        printf("✅ Results are correct.\n");
    else
        printf("❌ Mismatch found: %d errors.\n",
```

```
    errors);

        free(A); free(B); free(C_seq); free(C_parallel);
        return 0;
    }
```

4. Code Explanation

Code Part	Explanation
#include <omp.h>	Includes the OpenMP header file for parallel processing.
#pragma omp parallel for	Distributes loop iterations among threads for parallel execution.
omp_get_wtime()	Measures the elapsed wall-clock time for performance comparison.
malloc()	Dynamically allocates memory for large arrays.
Parallel vs Sequential	Compares execution time between standard and parallel processing.

5. Expected Output

```
Sequential addition time: 2.85432 seconds
Parallel addition time (OpenMP): 0.74319 seconds
✅ Results are correct.
```

6. Performance Comparison

Execution Type	Time (seconds)	Speedup
Sequential	2.85	1.0x
Parallel (OpenMP)	0.74	≈3.85x

7. How It Works

OpenMP divides loop iterations across all available CPU cores using threads. Each thread handles a subset of data simultaneously, which reduces total execution time. The runtime system manages thread creation, workload balancing, and synchronization. This demonstrates the efficiency of shared-memory parallelism in computational tasks.